

Stone Game III - Explanation

Idea: $dp[i]$ stores the maximum score difference (current player - opponent) starting from index i .

Transition:

```
dp[i] = max(  
    take1 - dp[i+1],  
    take2 - dp[i+2],  
    take3 - dp[i+3]  
)
```

Algorithm:

1. Initialize $dp[n]=0$.
2. Iterate from $n-1$ down to 0 .
3. Try taking 1,2,3 stones and update $dp[i]$.
4. If $dp[0]>0 \Rightarrow$ Alice, $dp[0]<0 \Rightarrow$ Bob, else Tie.

Java Code:

```
class Solution {  
    public String stoneGameIII(int[] stoneValue) {  
        int n = stoneValue.length;  
        int[] dp = new int[n + 1];  
        for (int i = n - 1; i >= 0; i--) {  
            dp[i] = Integer.MIN_VALUE;  
            int sum = 0;  
            for (int k = 0; k < 3 && i + k < n; k++) {  
                sum += stoneValue[i + k];  
                dp[i] = Math.max(dp[i], sum - dp[i + k + 1]);  
            }  
        }  
        if (dp[0] > 0) return "Alice";  
        if (dp[0] < 0) return "Bob";  
        return "Tie";  
    }  
}
```

Time: $O(n)$, Space: $O(n)$